

Egison Core

Syntax and Typing Rules

本仕様の機械化目標は、次を正確に主張できることにある。

The soundness of executable type inference for the Egison core is mechanized in Lean 4.

ここで soundness は、公開 executable inference が成功すれば、返した型に対する declarative expression typing が必ず構成できることを意味する。機械化した定理は後述する静的入力整形式性を追加前提にせず、completeness と full principality は含まない。

1 構文

1.1 Capability, target type, scheme

$\kappa ::= \bullet \mid \chi \mid \hat{s} \mid K \bar{\kappa} \mid (\kappa_1, \dots, \kappa_n)$	$\kappa : \text{Cap}$
$\tau ::= \alpha \mid \hat{a} \mid \mathbf{1} \mid \text{Int} \mid \text{Bool} \mid K \bar{\tau} \mid (\tau_1, \dots, \tau_n) \mid \tau_1 \rightarrow \tau_2$ $\mid \text{Matcher } \kappa \tau \mid \text{MatcherSlot } \kappa \tau$	$\tau : \text{Type}$
$\sigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \tau$	expression scheme
$\delta ::= (\kappa_1 \triangleright \tau_1), \dots, (\kappa_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau)$	pattern-function dual
$\varsigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \delta$	dual scheme
$\eta ::= \kappa \triangleright \tau$	hole descriptor.

図1 二 sort の型構文. χ と α は flexible meta-variable である. $\hat{s}$ と $\hat{a}$ の rigid skolem は、明示的な全称変数を型検査中だけ新鮮な固定定数として扱ったものであり、unification で別の capability / 型へ置換できない. $\text{mono } \tau$ は量化変数を一つも持たない単相 scheme を表し、環境から取り出して instantiate しても新しい変数を作らず同じ τ を返す。

1.2 Source forms

1.3 Contexts and judgments

$\hat{\Sigma} = (\hat{\Sigma}_D, \hat{\Sigma}_P, \hat{\Sigma}_F, \hat{\Sigma}_{prim}, \text{Obs}_{\hat{\Sigma}})$	freeze 済み canonical signature
$\Gamma ::= \emptyset \mid \Gamma, x : \sigma$	expression scheme context
$\Phi ::= \emptyset \mid \Phi, x : \text{Pattern}(\kappa \triangleright \tau)$	pattern-parameter dual context
$\Delta ::= \emptyset \mid \Delta, x : \tau$	monomorphic pattern-variable context.

$\Gamma(x)$, $\Phi(x)$, $\hat{\Sigma}_D(C)$ などは key が重複しない有限 map の lookup であり、key が存在しなければ失敗する。特に $\hat{\Sigma}_F$ の pattern-function 名は相異なる。“fresh” は現在の signature, context, 型, capability に自由出現しない同じ sort の変数を選ぶことを表す。scheme 内の量化 binder は局所的であり、無関係な fresh 選択の ambient scope には含まない。

$$\begin{aligned}
e &::= x \mid \lambda x. e \mid e_1 e_2 \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{fix } f x. e \mid n \\
&\quad \mid (e_1, \dots, e_n) \mid C \bar{e} \mid \text{op}(\bar{e}) \mid \text{something} \\
&\quad \mid \text{matcher } cls \\
&\quad \mid \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e \\
&\quad \mid \text{match } e_t \text{ as } e_m \text{ with } (p_i \rightarrow e_i)_i \\
p &::= \$x \mid _ \mid \#e \mid \tilde{x} \mid c \bar{p} \mid p_1 \& p_2 \mid p_1 \mid p_2 \mid f \bar{p} \mid (p_1, \dots, p_n) \\
pp &::= \$ \mid _ \mid \# \$x \mid c \bar{pp} \mid (pp_1, \dots, pp_n) \\
dp &::= \$x \mid _ \mid C \bar{dp} \mid (dp_1, \dots, dp_n) \\
cl &::= pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r \\
cls &::= [cl_i]_{i=1}^m \\
d &::= \text{def pattern } f(x_1 : \tau_1, \dots, x_n : \tau_n) := p \\
\ell &::= \text{none} \mid \chi \mid \hat{s} \\
ev &::= \text{unseen} \mid \text{known } \ell \mid K \bar{ev} \mid (ev_1, \dots, ev_n).
\end{aligned}$$

図2 c は pattern constructor, C は data constructor である。match は matchAll と head への surface elaboration であり, core primitive ではない。

$\widehat{\Sigma}; \Gamma \vdash e : \tau$	expression typing
$\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p : \text{Pattern}(\kappa \triangleright \tau); \Delta'$	pattern dual typing
$\widehat{\Sigma} \vdash pp : \text{PPPattern } \tau \rightsquigarrow \bar{\eta}; \Delta_{pp}$	primitive-pattern hole/capture extraction
$\widehat{\Sigma}_D \vdash dp : \text{PDPattern } \tau \rightsquigarrow \Gamma_{dp}$	data-pattern binding
$\widehat{\Sigma}; \Gamma \vdash cl : \text{Clause } \kappa \tau \rightsquigarrow ev$	matcher-clause typing and evidence
$\widehat{\Sigma}; \Gamma \vdash d \text{ ok} \Rightarrow (f : \varsigma)$	pattern-function definition

図3 Primary judgments. Δ と Γ_{dp} を Γ に入れるときは各 binding を $\text{mono } \tau$ として扱う。 $\boxplus$ は domain-disjoint union, pattern の Δ は左から右へ thread する。

2 補助定義

この節の「部分関数」は成功値または失敗を返す。 $f(x) = \text{some } y$ と $f(x) \Downarrow y$ はともに成功値が y であることを表し, $f(x) = \text{none}$ は失敗を表す。判断規則の premise に現れる部分関数が失敗した場合, その規則は適用できない。

2.1 Instantiation, generalization, paired substitution

Capability substitution C と target substitution T は別 sort であり, $S = (C, T)$ の作用を次で定める。

$$S\kappa = C\kappa, \quad S\tau = T(C\tau), \quad S\Gamma = \{x : S\sigma \mid x : \sigma \in \Gamma\}.$$

Paired substitution $S = (C, T)$ は, すべての $\alpha \in \text{dom}(T)$ について $C(T\alpha) = T\alpha$ を満たすとき range-fixed である。この条件は raw な matchCap / mguTy certificate と, Algorithm W が保持する最初の fresh allocation に対して用いる。一方, scheme binder の局所 mapping と, その結果に対する target specialization は, 名前の数値が偶然一致しても局所 binder が後段の型に入り込んだ capability を捕捉しないよう, 一つの global pair に潰さず順に適用する。

$$\begin{aligned}
\text{Inst}(\forall \bar{\chi}. \forall \bar{\alpha}. \tau) &= T_s(C_v \tau), \\
&\quad \text{dom}(C_v) \subseteq \bar{\chi}, \quad C_v \bar{\chi} \text{ capability variables,} \\
&\quad \text{dom}(T_s) \subseteq \bar{\alpha}, \\
\text{Inst}(\text{mono } \tau) &= \tau, \\
\text{Gen}(\widehat{\Sigma}, \Gamma, \tau) &= \forall (\text{fcv}(\tau) \setminus (\text{fcv}(\widehat{\Sigma}) \cup \text{fcv}(\Gamma))) : \text{Cap}. \\
&\quad \forall (\text{ftv}(\tau) \setminus (\text{ftv}(\widehat{\Sigma}) \cup \text{ftv}(\Gamma))) : \text{Type}. \tau, \\
\text{GenDual}(\widehat{\Sigma}, \Gamma, \delta) &= \forall (\text{fcv}(\delta) \setminus (\text{fcv}(\widehat{\Sigma}) \cup \text{fcv}(\Gamma))) : \text{Cap}. \\
&\quad \forall (\text{ftv}(\delta) \setminus (\text{ftv}(\widehat{\Sigma}) \cup \text{ftv}(\Gamma))) : \text{Type}. \delta.
\end{aligned}$$

Inst と InstDual の宣言的 relation は, capability binder を flexible capability variable へ置換し, その結果に target binder だけを support とする T_s を適用する. $C_v\chi$ は常に capability variable であり, $K\bar{\kappa}$ や capability product へ置換してはならない. T_s は capability variable を変更しない. この制限により, value producer の capability は利用側の consumer demand によって強化されない. T_s は capability 置換の後にだけ作用する. 従って, T_s が挿入した型の中に binder と同じ番号の自由 capability variable があっても, それを C_v で再び置換しない. この ordered binder-local semantics により, 量化 capability binder の variable-only instantiation と利用側から来る自由 capability を区別する. 宣言的 relation では binder の像の相異性や ambient freshness を要求しない. Algorithm W はより強い allocation witness として, 全 binder へ互いに異なる fresh variable を割り当て, solver が後続の target specialization を記録する. その allocation 証拠を忘却して target substitution を合成すれば, 上の宣言的 instance が得られる. InstDual は dual scheme の両 binder 列を対象にする. Gen と GenDual は signature と context の自由変数を量化しない. その型引数には prevailing paired substitution を適用し, 未解決の inference placeholder を含まない finalized type/capability を用いる. prevailing paired substitution とは, 同じ導出でそれ以前に得た全 constraint を解く累積的な一つの (C, T) である. Skolem は generalize しない. $\text{fcv}(X)$ と $\text{ftv}(X)$ はそれぞれ X に自由出現する flexible capability meta-variable と flexible target meta-variable の集合であり, scheme の binder に束縛された変数を含まない. $\text{dom}(X)$ は context または substitution X の定義域である.

Signature lookup の宣言的型付けは毎回上の ordered instance を使い, Algorithm W はそれを実装するとき fresh allocation witness を記録する.

$$\begin{aligned}\text{Inst}(\widehat{\Sigma}_D(C)) &= \tau_1 \times \cdots \times \tau_n \rightarrow \tau, \\ \text{Inst}(\widehat{\Sigma}_P(c)) &= (\tau_1, \dots, \tau_n) \Rightarrow_P \tau, \\ \text{InstDual}(\widehat{\Sigma}_F(f)) &= (\eta_1, \dots, \eta_n) \Rightarrow \eta, \\ \text{Inst}(\widehat{\Sigma}_{\text{prim}}(op)) &= \tau_1 \times \cdots \times \tau_n \rightarrow \tau.\end{aligned}$$

2.2 Equality, capability matching, signature operations

$$\begin{aligned}\text{mguTy}(\tau, \tau') &= T \quad \text{target-sort MGU,} \\ \text{mguCap}(\kappa, \kappa') &= C \quad \text{capability-sort MGU,} \\ \text{matchCap}(\kappa_m, \kappa_p) &= C \quad \text{dom}(C) \subseteq \text{fcv}(\kappa_p), \quad C\kappa_m = \kappa_m, \quad C\kappa_p = \kappa_m.\end{aligned}$$

matchCap は producer κ_m を rigid に読み, consumer demand κ_p の変数だけを解く most-general one-way match であり, 条件を満たす substitution がなければ失敗する. mguTy と mguCap はそれぞれの sort の決定的な部分関数であり, 成功時は most-general unifier を返す. constructor / arity の不一致, occurs check の失敗, または rigid skolem を解く必要がある場合は失敗する.

Frozen pattern signature が提供する target instance と, capability projection の整合判断を次で表す.

$$\pi_c = \text{Inst}(\widehat{\Sigma}_P(c)), \quad \text{CapCompatible}_{\widehat{\Sigma}}(\pi_c; \bar{\kappa}, \kappa), \quad \text{ty}_{\widehat{\Sigma}}(\pi_c; \bar{\tau}) \Downarrow \tau'.$$

π_c はこの二つの判断へ明示的に渡す同一の fresh pattern-constructor signature instance である. CapCompatible はその field に $\bar{\kappa}$ の canonical evidence を対応付け, result argument slot へ部分 evidence ev_p を投影した上で,

$$\text{merge}(ev_p, [\kappa]_{ev}) = \text{some } [\kappa]_{ev}$$

となるときに限り成立する. projection 自体が完全 capability を返すとは限らない. たとえば nullary nil は observable な list element parameter を unseen のまま残し, 周囲の最終 matcher capability がその位置を埋める. ty は $\bar{\tau}$ を field target と照合し, 同じ signature instance の result target を返す. いずれも arity, canonical former, または rigid constraint が一致しなければ失敗する. CapCompatible の projection は $\text{Obs}_{\widehat{\Sigma}}$ が公開する signature path だけをたどり, target substitution や expected target から capability を生成しない.

2.3 Matcher-clause metafunctions

$$\begin{aligned}
\text{prod}_0() &= (), \\
\text{prod}_1(\tau_1) &= \tau_1, \\
\text{prod}_k(\tau_1, \dots, \tau_k) &= (\tau_1, \dots, \tau_k) \quad (k \geq 2), \\
\text{decomposeME}(\cdot, 0) &= [], \\
\text{decomposeME}(M, 1) &= [M], \\
\text{decomposeME}((M_1, \dots, M_k), k) &= [M_1, \dots, M_k] \quad (k \geq 2).
\end{aligned}$$

上記以外の `decomposeME` は失敗する。() は nullary tuple value であり、その型も nullary product () である。1 とは同一視しない（この core では 1 の runtime constructor を持たない）。したがって $k = 0$ の matcher expression は () だけである。

Complete capability を partial evidence へ埋め込む canonical map は次である。

$$\begin{aligned}
[\bullet]_{ev} &= \text{known none}, & [\chi]_{ev} &= \text{known } \chi, \\
[\hat{s}]_{ev} &= \text{known } \hat{s}, & [K\bar{\kappa}]_{ev} &= K [\bar{\kappa}]_{ev}, \\
[(\kappa_1, \dots, \kappa_n)]_{ev} &= ([\kappa_1]_{ev}, \dots, [\kappa_n]_{ev}).
\end{aligned}$$

`clauseEvidenceŜ(pp; [κ1, ..., κk])` は、`pp` の hole を左から右へ並べた列と、それぞれの hole に実際に渡される next matcher の slot capability 列から、一 clause 分の partial evidence を返す決定的な部分関数である。bare-hole catch-all \$, wildcard _, value pattern # $\$x$ は root evidence を生成せず `unseen` を返す一方、constructor または tuple の内側にある \$ は対応する κ_i の canonical evidence を供給する。Tuple は component evidence をそのまま product node にし、constructor-headed `pp` は fresh/frozen signature の field type と result argument slot に沿って child evidence を投影する。このとき surface constructor 名ではなく signature の result former を evidence の root とする。hole 数, arity, canonical former, または同じ result slot へ届く evidence が exact に一致しなければ `none` を返す。target 型, expected 型, annotation は evidence の seed にしない。unseen は assignment を作らず、opaque / function / unobservable path は barrier として内部を観測しない。必要な observable path に既知の異なる head が現れた場合は、unseen へ弱めず失敗する。

`PPatCapsAtŜ(b, pp,  $\bar{\kappa}$ ,  $\kappa$ )` は、`pp` の hole 列を $\bar{\kappa}$ と左から右へ対応させ、その構造が最終 matcher capability κ と整合することを表す。nested hole はその位置の capability と同じでなければならない。constructor node は `CapCompatible`, tuple node は capability product を使う。root hole だけは catch-all として任意の一 hole capability を消費でき、wildcard と value pattern は hole を消費しない。

`ClausesTy` は actual clause list を順に検査し、一 clause につき一つの evidence を同じ順序で返す output-indexed judgment である。全 clause は同じ最終 capability と target を共有する。

$$\text{ClausesTy}(\hat{\Sigma}; \Gamma, [cl_i]_{i=1}^m, \kappa, \tau) \Downarrow [ev_i]_{i=1}^m \iff \forall i. \hat{\Sigma}; \Gamma \vdash cl_i : \text{Clause } \kappa \tau \rightsquigarrow ev_i.$$

いずれかの clause の PP extraction, next-matcher decomposition / slot typing, または一 clause 分の evidence projection が失敗すれば、この判断は成立しない。

2.4 Shape and coverage

Evidence の exact merge と shape inference は次で定める。

$$\begin{aligned}
\text{merge}(\text{unseen}, ev) &= ev & \text{merge}(ev, \text{unseen}) &= ev, \\
\text{merge}(\text{known } \ell, \text{known } \ell) &= \text{known } \ell, & \text{merge}(K\bar{e}, K\bar{f}) &= K \text{merge}(\bar{e}, \bar{f}), \\
\text{merge}((\bar{e}), (\bar{f})) &= (\text{merge}(\bar{e}, \bar{f})).
\end{aligned}$$

Constructor name, node kind, arity, leaf identity が一致しない merge は失敗する。上の式では成功時の some を省略している。列上の merge は同じ長さの列を pointwise に merge し、長さが異なるか一成分でも失敗すれば失敗する。

`mergeAll` は `unseen` を初期値として `evidence list` を `merge` で畳み込む決定的な部分関数であり、途中の `merge` が一度でも失敗すれば `none` を返す。

$$\text{inferShape}(Obs, \overline{ev}) = \begin{cases} \text{some } \bullet & \text{mergeAll}(\overline{ev}) = \text{some unseen}, \\ \text{finalize}_{Obs}(ev) & \text{mergeAll}(\overline{ev}) = \text{some } ev, ev \neq \text{unseen}, \\ \text{none} & \text{mergeAll}(\overline{ev}) = \text{none}. \end{cases}$$

finalize_{Obs} は partial evidence を complete capability へ変換する決定的な部分関数である。observable な `unseen` が残れば `none` を返し、unobservable な constructor parameter は $\bullet$ に canonicalize する。known `none`, known χ , known $\hat{s}$ はそれぞれ $\bullet$, χ , $\hat{s}$ に戻す。Constructor node は root former と同じ長さの frozen observability mask を用い、product component はすべて observable として再帰的に finalize する。未知の mask や arity 不一致は失敗する。

$$\text{ShapeCap}_{\widehat{\Sigma}}(\overline{ev}, \kappa) \iff \text{inferShape}(Obs_{\widehat{\Sigma}}, \overline{ev}) = \text{some } \kappa.$$

T-MATCHER は `ClausesTy` が actual clause list から返した同じ $\overline{ev}$ を `ShapeCap` へ渡すため、別の `evidence list` を選び直すことはできない。 $Obs_{\widehat{\Sigma}}(K)$ は canonical former K の parameter 数と同じ長さの Boolean mask を返し、各 parameter が pattern-constructor signature から capability として観測可能かを示す。この mask は freeze 前に全 signature から計算した least fixpoint であり、個々の matcher clause や target annotation には依存しない。

$$\begin{aligned} \text{CoverageOK}_{\widehat{\Sigma}}(cls, \bullet) &\iff \text{true}, \\ \text{CoverageOK}_{\widehat{\Sigma}}(cls, K\bar{\kappa}) &\iff \forall c \in \text{ctors}_{\widehat{\Sigma}_P}(K). \exists cl \in cls. cl.pp = c \$ \dots \$, \\ \text{CoverageOK}_{\widehat{\Sigma}}(cls, (\kappa_1, \dots, \kappa_n)) &\iff \exists cl \in cls. cl.pp = (\$, \dots, \$). \end{aligned}$$

`CoverageOK` のその他の root form は成立しない。Refinement は general clause に数えず、coverage は child capability へ再帰しない。 $\text{ctors}_{\widehat{\Sigma}_P}(K)$ は、fresh instantiate した result target の canonical root former が K である pattern constructor の有限集合であり、 $cl.pp$ は clause cl の primitive-pattern-pattern を取り出す selector である。Formal core では `CoverageOK` を matcher literal の必須条件とする。Egison 本体は同じ検査の失敗を warning として扱ってよいが、その診断方針は core の外側に置く。

2.5 Matcher well-formedness predicates

$$\begin{aligned} \text{CatchAllLast}(cls) &\iff \exists \text{prefix}, M, x, N. cls = \text{prefix} ++ [\$ \text{ as } M \text{ with } [\$x \rightarrow N]] \\ &\quad \wedge \forall cl \in \text{prefix}. cl.pp \neq \$, \\ \text{ArmExhaustive}_{\widehat{\Sigma}_D}(cls, \tau) &\iff \forall cl \in cls. \text{Exhaustive}_{\widehat{\Sigma}_D}(\text{arms}(cl), \tau), \\ \text{PPBindNodup}(cls) &\iff \forall cl \in cls. \text{NoDup}(\text{bindPP}(cl.pp)), \\ \text{ArmBindNodup}(cls) &\iff \forall cl \in cls. \forall dp \in \text{arms}(cl). (\text{NoDup}(\text{bindDP}(dp)) \\ &\quad \wedge \text{bindDP}(dp) \cap \text{bindPP}(cl.pp) = \emptyset). \end{aligned}$$

$\text{arms}(cl)$ は clause の $[dp_j \rightarrow N_j]_j$ から data pattern 列 $[dp_j]_j$ を順序を保って取り出す。 $\text{bindPP}(pp)$ と $\text{bindDP}(dp)$ はそれぞれ pp の $\# \$x$ と dp の $\$x$ が導入する変数名を左から右へ集め、`NoDup` はその列に重複がないことを表す。 $\text{Exhaustive}_{\widehat{\Sigma}_D}(\overline{dp}, \tau)$ は保守的で実行可能な checker である。core では $\overline{dp}$ に変数 pattern または wildcard が一つでもあれば成功し、それ以外は失敗する。したがって成功時には任意の runtime value を受理する具体的な arm が存在する。constructor の網羅性を推測する外部 oracle は用いない。

3 Expression typing

$$\begin{array}{c} \frac{\Gamma(x) = \sigma \quad \text{Inst}(\sigma) = \tau}{\widehat{\Sigma}; \Gamma \vdash x : \tau} \\ \text{[T-VAR]} \end{array} \quad \begin{array}{c} \frac{\widehat{\Sigma}; \Gamma, x : \text{mono } \tau_1 \vdash e : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \lambda x. e : \tau_1 \rightarrow \tau_2} \\ \text{[T-LAM]} \end{array} \quad \begin{array}{c} \frac{\widehat{\Sigma}; \Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \widehat{\Sigma}; \Gamma \vdash e_2 : \tau_1}{\widehat{\Sigma}; \Gamma \vdash e_1 e_2 : \tau_2} \\ \text{[T-APP]} \end{array} \quad \begin{array}{c} \frac{}{\widehat{\Sigma}; \Gamma \vdash n : \text{Int}} \\ \text{[T-LIT]} \end{array}$$

$$\begin{array}{c}
\frac{\widehat{\Sigma}; \Gamma \vdash e_1 : \tau_1 \quad \sigma = \text{Gen}(\widehat{\Sigma}, \Gamma, \tau_1) \quad \widehat{\Sigma}; \Gamma, x : \sigma \vdash e_2 : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \quad \text{[T-LET]} \qquad \frac{\widehat{\Sigma}; \Gamma, f : \text{mono}(\tau_1 \rightarrow \tau_2), x : \text{mono } \tau_1 \vdash e : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \text{fix } f.x.e : \tau_1 \rightarrow \tau_2} \quad \text{[T-FIX]} \\
\\
\frac{\widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash (e_1, \dots, e_n) : (\tau_1, \dots, \tau_n)} \quad \text{[T-TUPLE]} \\
\\
\frac{\text{Inst}(\widehat{\Sigma}_D(C)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash C e_1 \dots e_n : \tau} \quad \text{[T-CON]} \\
\\
\frac{\text{Inst}(\widehat{\Sigma}_{\text{prim}}(op)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash op(e_1, \dots, e_n) : \tau} \quad \text{[T-PRIM]} \qquad \frac{\alpha \text{ fresh in the target sort}}{\widehat{\Sigma}; \Gamma \vdash \text{something} : \text{Matcher} \bullet \alpha} \quad \text{[T-SOME]}
\end{array}$$

4 Pattern typing and pattern functions

$$\begin{array}{c}
\frac{x \notin \text{dom}(\Delta) \quad \chi, \alpha \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \$x : \text{Pattern}(\chi \triangleright \alpha); \Delta, x : \alpha} \quad \text{[PAT-VAR]} \qquad \frac{\chi, \alpha \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash _ : \text{Pattern}(\chi \triangleright \alpha); \Delta} \quad \text{[PAT-WILD]} \\
\\
\frac{\widehat{\Sigma}; \Gamma, \text{mono } \Delta \vdash e : \tau \quad \chi \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \#e : \text{Pattern}(\chi \triangleright \tau); \Delta} \quad \text{[PAT-VALUE]} \qquad \frac{\Phi(x) = \text{Pattern}(\kappa \triangleright \tau)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \widetilde{x} : \text{Pattern}(\kappa \triangleright \tau); \Delta} \quad \text{[PAT-EMBED]} \\
\\
\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash (p_1, \dots, p_n) : \text{Pattern}((\kappa_1, \dots, \kappa_n) \triangleright (\tau_1, \dots, \tau_n)); \Delta_n} \quad \text{[PAT-TUPLE]} \\
\\
\frac{\pi_c = \text{Inst}(\widehat{\Sigma}_P(c)) = (\rho_1, \dots, \rho_n) \Rightarrow_P \rho \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n) \quad \text{CapCompatible}_{\widehat{\Sigma}}(\pi_c; \bar{\kappa}, \kappa) \quad \text{ty}_{\widehat{\Sigma}}(\pi_c; \bar{\tau}) \Downarrow \tau}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash c p_1 \dots p_n : \text{Pattern}(\kappa \triangleright \tau); \Delta_n} \quad \text{[PAT-CON]} \\
\\
\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 : \text{Pattern}(\kappa \triangleright \tau); \Delta_1 \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_1 \vdash p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta_2}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 \& p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta_2} \quad \text{[PAT-AND]} \\
\\
\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 : \text{Pattern}(\kappa \triangleright \tau); \Delta' \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta'}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 \mid p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta'} \quad \text{[PAT-OR]} \\
\\
\frac{\text{InstDual}(\widehat{\Sigma}_F(f)) = (\kappa_1 \triangleright \tau_1), \dots, (\kappa_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau) \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash f p_1 \dots p_n : \text{Pattern}(\kappa \triangleright \tau); \Delta_n} \quad \text{[PAT-APP]}
\end{array}$$

$$\begin{array}{c}
\widehat{\Sigma}_F(f) = \varsigma_f \quad f \notin \text{pcalls}(p_{\text{body}}) \quad \chi_1, \dots, \chi_n \text{ fresh} \\
\Phi' = \emptyset, x_1 : \text{Pattern}(\chi_1 \triangleright \tau_1), \dots, x_n : \text{Pattern}(\chi_n \triangleright \tau_n) \quad \widehat{\Sigma}; \Gamma; \Phi'; \emptyset \vdash p_{\text{body}} : \text{Pattern}(\kappa \triangleright \tau); \Delta \\
\widehat{x}_1, \dots, \widehat{x}_n \text{ occur exactly once, in declaration order, outside or-alternatives} \\
\varsigma_f = \text{GenDual}(\widehat{\Sigma} \setminus f, \Gamma, (\chi_1 \triangleright \tau_1), \dots, (\chi_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau)) \\
\hline
\widehat{\Sigma}; \Gamma \vdash \text{def pattern } f(x_1 : \tau_1, \dots, x_n : \tau_n) := p_{\text{body}} \text{ ok} \Rightarrow (f : \varsigma_f)
\end{array}$$

[PATFUN-DEF]

ここで $\widehat{\Sigma} \setminus f$ は、有限 map $\widehat{\Sigma}_F$ から一意な f の entry だけを除いた signature である。本体は freeze 済みの完全な $\widehat{\Sigma}$ で検査し、自身の scheme だけを generalization の ambient free-variable 集合から除外する。pcalls は pattern の全子孫を走査し、value pattern 内の式、その `matchAll` pattern, matcher clause の next と各 arm body も含めて自己 papp を拒否する。従って他の pattern function への参照は許すが、直接自己再帰は許さない。

5 Match and MatcherSlot

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_t : \tau_t \quad \widehat{\Sigma}; \Gamma; \emptyset; \emptyset \vdash p : \text{Pattern}(\kappa_p \triangleright \tau_t); \Delta \quad \widehat{\Sigma}; \Gamma \vdash e_m : \text{MatcherSlot } \kappa_p \tau_t \quad \widehat{\Sigma}; \Gamma, \text{mono } \Delta \vdash e : \tau_r}{\widehat{\Sigma}; \Gamma \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e : \text{List } \tau_r}$$

[T-MATCHALL]

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_t : \tau_t \quad \widehat{\Sigma}; \Gamma; \emptyset; \emptyset \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_t); \Delta_i \quad (\forall i) \quad \widehat{\Sigma}; \Gamma \vdash e_m : \text{MatcherSlot } \kappa_i \tau_t \quad (\forall i) \quad \widehat{\Sigma}; \Gamma, \text{mono } \Delta_i \vdash e_i : \tau_r \quad (\forall i)}{\widehat{\Sigma}; \Gamma \vdash \text{match } e_t \text{ as } e_m \text{ with } (p_i \rightarrow e_i)_i : \tau_r}$$

[T-MATCH (DERIVED)]

$$\frac{\text{matchCap}(\kappa_m, \kappa_p) = C \quad \widehat{\Sigma}; \Gamma \vdash e : \text{Matcher } \kappa_m \tau_m \quad \text{mguTy}(C\tau_m, C\tau_t) = T \quad S = (C, T) \quad C(T\alpha) = T\alpha \quad (\alpha \in \text{dom}(T))}{\widehat{\Sigma}; S\Gamma \vdash e : \text{MatcherSlot } C\kappa_p S\tau_t}$$

[COERCE-MATCHER-TO-SLOT]

$$\frac{\text{mguCap}(\kappa', \kappa) = C \quad \widehat{\Sigma}; \Gamma \vdash e : \text{MatcherSlot } \kappa' \tau' \quad \text{mguTy}(C\tau', C\tau) = T \quad S = (C, T) \quad C(T\alpha) = T\alpha \quad (\alpha \in \text{dom}(T))}{\widehat{\Sigma}; S\Gamma \vdash e : \text{MatcherSlot } S\kappa S\tau}$$

[CHECK-SLOT-TO-SLOT]

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_i : \text{Matcher } \kappa_i \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash (e_1, \dots, e_n) : \text{Matcher } (\kappa_1, \dots, \kappa_n) (\tau_1, \dots, \tau_n)} \quad \frac{\widehat{\Sigma}; \Gamma \vdash e : (\text{MatcherSlot } \kappa_1 \tau_1, \dots, \text{MatcherSlot } \kappa_n \tau_n)}{\widehat{\Sigma}; \Gamma \vdash e : \text{MatcherSlot } (\kappa_1, \dots, \kappa_n) (\tau_1, \dots, \tau_n)}$$

[COERCE-TUPLE-MATCHER] [COERCE-SLOT-TUPLE]

6 Matcher literals

6.1 Primitive pattern patterns

$$\frac{\chi \text{ fresh}}{\widehat{\Sigma} \vdash \$: \text{PPPattern } \tau \rightsquigarrow [\chi \triangleright \tau]; \emptyset} \quad \frac{}{\widehat{\Sigma} \vdash _ : \text{PPPattern } \tau \rightsquigarrow []; \emptyset} \quad \frac{}{\widehat{\Sigma} \vdash \# \$ x : \text{PPPattern } \tau \rightsquigarrow []; x : \tau}$$

[PP-HOLE] [PP-WILD] [PP-VAL]

$$\frac{\text{Inst}(\widehat{\Sigma}_P(c)) = (\tau_1, \dots, \tau_n) \Rightarrow_P \tau \quad \widehat{\Sigma} \vdash pp_i : \text{PPPattern } \tau_i \rightsquigarrow \bar{\eta}_i; \Delta_i \quad (1 \leq i \leq n) \quad \bar{\eta} = \bar{\eta}_1 ++ \dots ++ \bar{\eta}_n}{\widehat{\Sigma} \vdash c pp_1 \dots pp_n : \text{PPPattern } \tau \rightsquigarrow \bar{\eta}; \Delta_1 \uplus \dots \uplus \Delta_n} \quad [\text{PP-CON}]$$

$$\frac{\widehat{\Sigma} \vdash pp_i : \text{PPPattern } \tau_i \rightsquigarrow \bar{\eta}_i; \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma} \vdash (pp_1, \dots, pp_n) : \text{PPPattern } (\tau_1, \dots, \tau_n) \rightsquigarrow \bar{\eta}_1 ++ \dots ++ \bar{\eta}_n; \Delta_1 \uplus \dots \uplus \Delta_n} \quad [\text{PP-TUPLE}]$$

6.2 Primitive data patterns

$$\frac{}{\widehat{\Sigma}_D \vdash \$x : \text{PDPattern } \tau \rightsquigarrow x : \tau} \quad [\text{PD-VAR}] \qquad \frac{}{\widehat{\Sigma}_D \vdash _ : \text{PDPattern } \tau \rightsquigarrow \emptyset} \quad [\text{PD-WILD}]$$

$$\frac{\text{Inst}(\widehat{\Sigma}_D(C)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}_D \vdash dp_i : \text{PDPattern } \tau_i \rightsquigarrow \Gamma_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}_D \vdash C dp_1 \dots dp_n : \text{PDPattern } \tau \rightsquigarrow \Gamma_1 \uplus \dots \uplus \Gamma_n} \quad [\text{PD-CON}]$$

$$\frac{\widehat{\Sigma}_D \vdash dp_i : \text{PDPattern } \tau_i \rightsquigarrow \Gamma_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}_D \vdash (dp_1, \dots, dp_n) : \text{PDPattern } (\tau_1, \dots, \tau_n) \rightsquigarrow \Gamma_1 \uplus \dots \uplus \Gamma_n} \quad [\text{PD-TUPLE}]$$

6.3 Clauses and matcher construction

$$\frac{\begin{array}{l} \widehat{\Sigma} \vdash pp : \text{PPPattern } \tau \rightsquigarrow [(\kappa_1 \triangleright \tau_1), \dots, (\kappa_k \triangleright \tau_k)]; \Delta_{pp} \\ \text{PPatCapsAt}_{\widehat{\Sigma}}(\text{root}, pp, [\kappa_1, \dots, \kappa_k], \kappa) \quad \text{decomposeME}(M, k) = [M_1, \dots, M_k] \\ \widehat{\Sigma}; \Gamma \vdash M_l : \text{MatcherSlot } \kappa_l \tau_l \quad (1 \leq l \leq k) \quad \widehat{\Sigma}_D \vdash dp_j : \text{PDPattern } \tau \rightsquigarrow \Gamma_j \quad (1 \leq j \leq r) \\ \widehat{\Sigma}; \Gamma, \text{mono } \Delta_{pp}, \text{mono } \Gamma_j \vdash N_j : \text{List prod}_k(\tau_1, \dots, \tau_k) \quad (1 \leq j \leq r) \quad \text{clauseEvidence}_{\widehat{\Sigma}}(pp; [\kappa_1, \dots, \kappa_k]) = \text{some } ev \end{array}}{\widehat{\Sigma}; \Gamma \vdash (pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r) : \text{Clause } \kappa \tau \rightsquigarrow ev} \quad [\text{CLAUSE-TY}]$$

$$\frac{\begin{array}{l} \text{ClausesTy}(\widehat{\Sigma}; \Gamma, cls, \kappa, \tau) \Downarrow \bar{ev} \quad \text{ShapeCap}_{\widehat{\Sigma}}(\bar{ev}, \kappa) \quad \text{CatchAllLast}(cls) \\ \text{ArmExhaustive}_{\widehat{\Sigma}_D}(cls, \tau) \quad \text{PPBindNodup}(cls) \quad \text{ArmBindNodup}(cls) \quad \text{CoverageOK}_{\widehat{\Sigma}}(cls, \kappa) \end{array}}{\widehat{\Sigma}; \Gamma \vdash \text{matcher } cls : \text{Matcher } \kappa \tau} \quad [\text{T-MATCHER}]$$

7 機械化する定理の境界

7.1 安全な instance と後続輸送

Inst が使う宣言的 instance と, Algorithm W の allocation witness を区別する. 宣言的な C_v と T_s はそれぞれの scheme binder 上だけに support を持ち, C_v は各 binder を capability variable へ写す. この層では像の相異性, ambient freshness, range-fixedness を使わない. これに対し W の (C_f, T_f) は, 像が互いに異なり ambient scope に対して fresh であることと range-fixedness を記録する. ここで ambient scope は signature と context の自由変数が

らなり、別の scheme が局所的に束縛する同名の数値 identifier は含めない。W の witness からこれらの allocation 証拠を忘却すると宣言的 instance が得られる。

Target specialization T_s は capability instance に対して後から作用し、その像に C_v を再適用する global 作用にはまともでない。証明中で shape evidence を輸送する宣言的な後続 post も capability variable を capability variable にだけ写す。その層では単射性、像の相異性、ambient freshness を要求しない。Algorithm W の後続 chain は有限 support と fresh allocation を記録するより強い witness であり、それを忘却して宣言的 post を得る。各 post は現在の導出へ順に作用するため、前の局所 binder と後から挿入された自由変数を混同しない。

これに対して、raw な matchCap / mguTy certificate は生成時の正規化済み index と局所 cut とともに保持する。成功 run の終端までに得た solver delta は、後段の capability action を前段の target range 内部にも適用する cross-sort-aware な逐次作用で replay する。終端 validator は、記録した局所 cut と唯一の終端 cut の間で必要な alignment と restricted post を検査する。従って誤った componentwise 可換性を仮定せず、solver を別の入力で再実行する必要もない。中間の各 stop を追加の公開前提として量化しない。したがって次が成り立つ。

整型な value producer を scheme から取り出した後、consumer demand, expected target, annotation, または再帰 self-reference を根拠として、その intrinsic capability を構造的に強化することはできない。

primitive-pattern の hole capability は value producer ではなく clause evidence の consumer placeholder である。これらは matcher literal 全体で一つの prevailing substitution により解決し、actual clause evidence と CoverageOK がその構造を根拠づける。両者を同じ instantiation 経路で扱わない。

Let specialization では、bound expression の検査直後に得た context Γ_0 、型 τ_0 、scheme $\sigma_0 = \text{Gen}(\widehat{\Sigma}, \Gamma_0, \tau_0)$ を trace に記録する。body の constraint をすべて反映した終端 substitution S_r に対して validator が検査するのは、次の終端可換式である。

$$S_r \sigma_0 = \text{Gen}(\widehat{\Sigma}, S_r \Gamma_{\text{raw}}, S_r \tau_{\text{raw}}).$$

従って、body の検査後も Γ_0 、 τ_0 、 σ_0 が構文的に不変であるとは仮定しない。外側の instance が導入した自由変数と内側の量化 binder の数値 identifier が一致する場合には、内側の binder を target ambient の外へ局所的に freshen してから輸送する。この freshening は source-level の導出輸送補題である。公開 inference の T-LET bridge は上の終端可換式を validator で直接検査して再構成し、型付け導出そのものを premise とする oracle は置かない。

7.2 検証付き Algorithm W と再構成

raw traversal inferRaw は expression, pattern, primitive pattern, data pattern, clause, slot check を有限な fuel 再帰で実行し、成功時には全 solver step と source event を trace に保持する。fresh value instance の capability variable は protected producer として記録し、全 solver delta がそれらを固定することを実行時にも監査する。matcher literal の成功には CoverageOK, catch-all, arm exhaustiveness, binder 線形性の各 checker の成功を要求する。

Pattern constructor の capability inference は projection の unseen 位置を fresh capability variable で埋めた候補を作り、後続 constraint で最終 matcher capability と照合する。各 clause が保持する raw hole dual は、literal 全体の prevailing substitution が確定した後に一度だけ zonk する。その時点で全 clause evidence を再計算し、PPatCapsAt の executable checker も最終 capability に対して実行する。途中の clause で計算した古い evidence を finalization に再利用しない。これらは target や expected type を evidence の seed にしない。

各 nested pattern はそれ自身の fresh raw index を持ち、solver は親が要求する index と終端 substitution 適用後に一致させる。従って再構成では、生成時 certificate を残す raw provenance と、実際の終端 index で構造を追う terminal derivation を分ける。子の raw index と親の raw field index が構文的に同一であるという、Algorithm W が保証しない条件は要求しない。source typing と runtime proof は terminal derivation の射影だけを利用する。

公開する infer は raw traversal の成功をそのまま公開せず、有限の終端代数 validator wBridgeCheck を通した結果だけを返す。概念的な定義は次である。

$$\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{bind}(\text{inferRaw}(\widehat{\Sigma}, \Gamma, e), \lambda r. \\ \text{if } \text{wBridgeCheck}(\widehat{\Sigma}, r) \text{ then some } r \text{ else none}).$$

`wBridgeCheck` は signature と有限な result / trace だけを走査し、記録した supply に基づく binder-local instance, slot / type / dual alignment, lookup 時の fresh instance と終端 instance の対応, matcher finalization, 終端 let-generalization, および concrete coverage/exhaustiveness check を検査する. source typing judgment やその導出を検査対象・保存データとして持たない. validator の成功から `WBridgeWF` を Lean 内で構成し, raw traversal に対する再構成補題へ渡す. solver replay 自体は cross-sort-aware な逐次合成から無条件に導かれる. 元の context / expression と result / trace の対応は raw traversal の成功等式が与える. また公開 soundness の経路では, 生成時の `FreshInstAt` を後続 solver cut ごとに輸送せず, 終端の binder-local `ValueFlowInst` / `Inst` を有限 checker で直接構成する. 従って `WBridgeWF` は実装内部の中間証明であり, 公開 soundness theorem の caller が与える premise ではない. raw traversal が成功したが終端 validator が失敗した場合, 公開 infer は失敗を返す. この filter が全 typable term または全 raw 成功を受理するという completeness は主張しない.

意図する frozen Egison core 入力の静的境界 `InferenceInputWF`($\widehat{\Sigma}, \Gamma$) は, $\widehat{\Sigma}$ の arm-exhaustiveness 関数が concrete `basicArmExhaustive` checker であること, および signature と context の lookup から選択可能な各 scheme の capability binder 列と target binder 列がそれぞれ重複を持たないことを要求する. これは runtime safety 用の `FrozenSigWF` とは別の静的境界である. ただし fail-closed な validator が成功結果に必要な代数条件をすべて検査するため, この境界は公開 soundness 定理の前提ではない. $S_r = r.\text{state}.\text{prevailing}$ とおくと, 機械化した公開 soundness は次の, より強い形である.

$$\frac{\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{some } r}{\widehat{\Sigma}; S_r, \Gamma \vdash e : r.\text{resolvedTarget}}$$

ここで $r.\text{resolvedTarget}$ は S_r を raw result target に適用した型である. raw traversal と validator はともに停止する executable function であり, 定理の premise には caller-supplied bridge, 型付け oracle, completeness, full principal-type theorem を含めない.

singleton direct-self の `fix` では, 再帰 binder は RHS 内で一つの monotype placeholder を共有する. binder の自由出現は application head に直接現れるものだけを認め, alias, 引数としての受渡し, 相互再帰, 高階 origin は fail closed とする. actual clause evidence 以外の self-reference や demand は `ShapeCap` の seed にしない.

7.3 Concrete runtime safety

runtime matcher value は source の全 clause 列と, 実行中の current clause/arm cursor を別々に保持する. ただし公開 matching-state 境界では, 値・tuple / constructor の内部・closure の captured environment・matcher captured environment・内部 node と stack のすべてについて cursor が original clause 列にあることを `Pristine` 不変条件として要求する. suffix cursor は一つの `MAtom` 導出の内部だけで使い, successor state へは流出させない. 整型 matcher value からは元の actual clause typing, evidence, `CoverageOK` を復元できる. matching state typing は atom ごとの prevailing substitution, pattern-function node の固定 parameter context, 残余 actual argument, 内部 stack, および terminal substitution の binding 順序を追跡する.

Ordered clause dispatch の内部では, original clause 列から現在位置までの失敗 prefix を `DispatchTrace` が保持する. freeze 済み signature は, 同じ outer capability と pattern-constructor entry に compatible な child capability 列が一意であること, および各 named pattern constructor が coverage index の canonical former と arity に登録されていることを要求する. これにより最後の bare-hole catch-all まで到達した structured pattern は, 先行する対応 constructor clause の失敗事実と矛盾し, stuck にならない.

値パターン式 $\#e$ は同じ原子の先行 hole binding ではなく, 原子へ入る時点の context で型付くことを `CaptureAdm` とする. また, 一段の clause dispatch が実際に必要とする body / next-matcher の局所評価結果だけを `StepReady`

とする。後者は一般の program termination や既成の Step 導出を仮定しない。

freeze 済み signature の整形形式性と source/runtime pattern-function signature の一致の下で, Lean の concrete judgments 上に次を証明する。

1. pristine な typed environment ρ での $\text{Eval } \rho \ e \ v$ は HasTy を ValueTy へ保存する。
2. 整型 matching state の一段 Step の全 successor は整型である。
3. 非終端の整型 state は, 局所証拠 StepReady の下で一段進む。空 successor 列は正当な match failure であり stuck ではない。
4. 一段保存の反復により, 到達可能な全 state は整型である。
5. 空 stack へ到達した成功 branch の substitution は, source pattern が生成した binding context で型付く。
6. matcher consistency は前項を初期 atom からの到達へ適用した系として得る。

最後に W の再構成定理をこの declarative safety package へ接続する。CaptureAdm と局所 StepReady 以外に, 任意の capability 輸送, 抽象 runtime oracle, または型付け済み外部 list / multiset matcher を前提として残さない。